

BYOL From Scratch

A Complete Technical Implementation Guide

Self-Supervised Visual Representation Learning with Custom CUDA Kernels, Memory Optimisations, and
Controlled Ablations

Hardware: NVIDIA RTX 3050 Laptop GPU | Dataset: STL-10 | Backbones: ResNet-18 & ViT-Tiny | Duration: 8–10
weeks

Project Overview

This project is a from-scratch implementation of Bootstrap Your Own Latent (BYOL), a self-supervised visual representation learning method. Every component is written independently: two backbone architectures (ResNet-18 and ViT-Tiny), projector, predictor, momentum encoder, augmentation pipeline, custom CUDA kernels, and memory optimisations. The goal is deep demonstrable understanding of modern DL training, not benchmark performance.

Backbones	ResNet-18 AND ViT-Tiny — both implemented from scratch, compared directly	
Dataset	STL-10 — 100k unlabelled 96×96 images, purpose-built for self-supervised learning	
Hardware	RTX 3050 Laptop (Ampere sm_86, 4–6 GB VRAM)	
Cloud wrap-up	Vast.ai or Lambda Labs for final eval (~2–4 hrs, ~\$2–5 total)	
Memory techniques	AMP FP16, gradient accumulation, CPU-offload Adam, gradient checkpointing	
CUDA kernels	Fused Gaussian blur (both backbones) + fused batch norm (ResNet) / layer norm (ViT)	
Evaluation	kNN accuracy + linear probe every N steps; t-SNE at checkpoints; collapse detection	
Ablations	6 controlled experiments verifying theoretical claims of the BYOL paper	
Green label = ResNet-18 specific Amber label = ViT-Tiny specific No label = applies to both backbones		

BYOL Architecture

BYOL wraps any backbone in an online/target pair. The backbone choice (ResNet-18 or ViT-Tiny) plugs into the same outer structure — projector, predictor, EMA updater, loss — which is identical for both. The differences are entirely inside the backbone and in which norm kernel you optimise.

Component	Online Network	Target Network
Backbone (f)	ResNet-18 or ViT-Tiny — receives gradients	EMA copy — no gradients
Projector (g)	MLP: feat_dim $\rightarrow$ 4096 $\rightarrow$ 256, BN/LN + ReLU	Identical MLP, frozen
Predictor (q)	MLP: 256 $\rightarrow$ 4096 $\rightarrow$ 256, BN/LN + ReLU	ABSENT — prevents collapse
Update	Back-prop + Adam (CPU-offloaded)	EMA of online weights
Loss	Normalised MSE, symmetric across both views	—

Property	ResNet-18	ViT-Tiny
Parameters	~11M	~5–6M
Feature dim	512	192
Normalisation	Batch norm	Layer norm
Attention	None	Multi-head self-attention
CUDA kernel target	Fused batch norm	Fused layer norm + fused attention
Collapse detection	Easier — more stable training	Harder — needs careful LR warmup
Modern relevance	Solid, well understood	State-of-the-art (DINO, MAE lineage)
Implementation order	Build first — simpler to debug	Build second — swap backbone module

Do not import torchvision or timm. Write both backbones yourself. Start with ResNet-18, verify it works on a supervised task, then implement ViT-Tiny. Both must expose the same interface: `forward(x) -> feature_vector`.

1a. ResNet-18 BasicBlock

■ ResNet-18

```
class BasicBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_ch))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        return F.relu(out)

class ResNet18(nn.Module):
    def __init__(self):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 3, stride=1, padding=1, bias=False), # 3x3 for 96x96
            nn.BatchNorm2d(64), nn.ReLU(inplace=True))
        self.layer1 = self._make(64, 64, 2, stride=1)
        self.layer2 = self._make(64, 128, 2, stride=2)
        self.layer3 = self._make(128, 256, 2, stride=2)
        self.layer4 = self._make(256, 512, 2, stride=2)
        self.pool = nn.AdaptiveAvgPool2d(1)

    def _make(self, in_ch, out_ch, n, stride):
        layers = [BasicBlock(in_ch, out_ch, stride)]
        for _ in range(1, n):
            layers.append(BasicBlock(out_ch, out_ch))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer4(self.layer3(self.layer2(self.layer1(x))))
        return self.pool(x).flatten(1) # [B, 512]
```

Use a 3x3 stem conv (not 7x7) for STL-10's 96x96 input — a 7x7/stride-2 stem loses too much spatial resolution at this size.

1b. ViT-Tiny Building Blocks

■ ViT-Tiny

```

class PatchEmbed(nn.Module):
    def __init__(self, img_size=96, patch_size=8, in_ch=3, embed_dim=192):
        super().__init__()
        self.n_patches = (img_size // patch_size) ** 2  # 144 patches for 96x96, p=8
        self.proj = nn.Conv2d(in_ch, embed_dim,
                               kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        x = self.proj(x)  # [B, embed_dim, H/P, W/P]
        x = x.flatten(2)  # [B, embed_dim, n_patches]
        return x.transpose(1, 2)  # [B, n_patches, embed_dim]

class ViTBlock(nn.Module):
    def __init__(self, embed_dim=192, n_heads=3, mlp_ratio=4.0, dropout=0.0):
        super().__init__()
        self.norm1 = nn.LayerNorm(embed_dim)
        self.attn = nn.MultiheadAttention(embed_dim, n_heads,
                                           dropout=dropout, batch_first=True)

        self.norm2 = nn.LayerNorm(embed_dim)
        mlp_dim = int(embed_dim * mlp_ratio)
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, mlp_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(mlp_dim, embed_dim),
            nn.Dropout(dropout))

    def forward(self, x):
        attn_out, _ = self.attn(self.norm1(x), self.norm1(x), self.norm1(x))
        x = x + attn_out
        x = x + self.mlp(self.norm2(x))
        return x

class ViTTiny(nn.Module):
    def __init__(self, img_size=96, patch_size=8, embed_dim=192,
                 depth=12, n_heads=3):
        super().__init__()
        self.patch_embed = PatchEmbed(img_size, patch_size, embed_dim=embed_dim)
        n_patches = self.patch_embed.n_patches
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(torch.zeros(1, n_patches + 1, embed_dim))
        self.blocks = nn.Sequential(*[ViTBlock(embed_dim, n_heads) for _ in range(depth)])
        self.norm = nn.LayerNorm(embed_dim)
        self._init_weights()

    def _init_weights(self):
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)
        cls = self.cls_token.expand(B, -1, -1)
        x = torch.cat([cls, x], dim=1) + self.pos_embed
        x = self.norm(self.blocks(x))
        return x[:, 0]  # CLS token -> [B, 192]

```

ViT-Tiny config for STL-10: embed_dim=192, depth=12, n_heads=3, patch_size=8. This gives 144 patches per image and ~5.7M parameters total.

1c. Unified Backbone Interface

Both backbones must share the same forward signature so the BYOL wrapper is backbone-agnostic. Verify this with a shape test before proceeding:

```
x = torch.randn(4, 3, 96, 96)
assert ResNet18()(x).shape == (4, 512), "ResNet feature dim wrong"
assert ViTTiny()(x).shape == (4, 192), "ViT feature dim wrong"
# Projector input dim must match backbone output dim
# Pass feat_dim explicitly to MLP rather than hardcoding 512
```

1d. STL-10 Data Pipeline (both backbones)

```
class STL10Unlabeled(Dataset):
    def __init__(self, data_path, view1_transform, view2_transform):
        self.data = np.load(data_path) # [100000, 3, 96, 96]
        self.t1, self.t2 = view1_transform, view2_transform

    def __getitem__(self, idx):
        img = Image.fromarray(self.data[idx].transpose(1,2,0))
        return self.t1(img), self.t2(img)

    def __len__(self): return len(self.data)

def byol_transform(view=1, size=96):
    return transforms.Compose([
        transforms.RandomResizedCrop(size, scale=(0.08, 1.0)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomApply([transforms.ColorJitter(0.4,0.4,0.2,0.1)], p=0.8),
        transforms.RandomGrayscale(p=0.2),
        transforms.RandomApply([transforms.GaussianBlur(23)],
                                p=1.0 if view==1 else 0.1), # asymmetric
        transforms.RandomSolarize(128, p=0.0 if view==1 else 0.2),
        transforms.ToTensor(),
        transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225]),
    ])

loader = DataLoader(
    STL10Unlabeled(path, byol_transform(1), byol_transform(2)),
    batch_size=64, shuffle=True, num_workers=4,
    pin_memory=True, prefetch_factor=2, persistent_workers=True)
```

1e. Phase 1 Deliverables

- ResNet-18 and ViT-Tiny both pass shape assertions
- Supervised CIFAR-10 smoke test: loss drops in 100 steps for BOTH backbones
- STL-10 DataLoader produces correct two-view pairs at correct shapes
- GPU utilisation >90% during data loading (profile with nvidia-smi)
- Two-view augmentation asymmetry verified by saving sample image pairs

The BYOL outer shell is identical regardless of backbone. Pass `feat_dim` as a constructor argument so the same class works for both ResNet-18 (512) and ViT-Tiny (192).

2a. Projector and Predictor MLPs

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim, use_bn=True):
        super().__init__()
        # use_bn=True for ResNet (BN is stable with larger batches)
        # use_bn=True also works for ViT – LN alternative also fine
        norm = nn.BatchNorm1d(hidden_dim) if use_bn else nn.LayerNorm(hidden_dim)
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim), norm,
            nn.ReLU(inplace=True),
            nn.Linear(hidden_dim, out_dim))

    def forward(self, x): return self.net(x)
```

2b. BYOL Module

```
class BYOL(nn.Module):
    def __init__(self, backbone, feat_dim, proj_dim=256, hidden=4096):
        super().__init__()
        self.online_enc = backbone
        self.online_proj = MLP(feat_dim, hidden, proj_dim)
        self.predictor = MLP(proj_dim, hidden, proj_dim)
        self.target_enc = copy.deepcopy(backbone)
        self.target_proj = copy.deepcopy(self.online_proj)
        for p in (*self.target_enc.parameters(),
                  *self.target_proj.parameters()):
            p.requires_grad = False

    def forward(self, v1, v2):
        z1 = self.predictor(self.online_proj(self.online_enc(v1)))
        z2 = self.predictor(self.online_proj(self.online_enc(v2)))
        with torch.no_grad():
            t1 = self.target_proj(self.target_enc(v1))
            t2 = self.target_proj(self.target_enc(v2))
        return 0.5 * (self._loss(z1, t2) + self._loss(z2, t1))

    @staticmethod
    def _loss(x, y):
        x = F.normalize(x, dim=-1)
        y = F.normalize(y, dim=-1)
        return 2 - 2*(x*y).sum(dim=-1).mean()

# Instantiation
resnet_byol = BYOL(ResNet18(), feat_dim=512)
vit_byol = BYOL(ViTTiny(), feat_dim=192)
```

2c. EMA Updater with Cosine Momentum Schedule

Momentum increases from 0.996 toward 1.0 on a cosine schedule. Most reimplementations use a fixed momentum. The schedule is in Appendix B of the paper and measurably affects representation quality.

```

class EMAUpdater:
    def __init__(self, online, target, base_tau=0.996, total_steps=100_000):
        self.online, self.target = online, target
        self.base_tau = base_tau
        self.total_steps = total_steps
        self.step = 0

    def update(self):
        tau = 1 - (1 - self.base_tau) * (
            math.cos(math.pi * self.step / self.total_steps) + 1) / 2
        for op, tp in zip(self.online.parameters(),
                          self.target.parameters()):
            tp.data = tau * tp.data + (1 - tau) * op.data
        self.step += 1
        return tau    # log this – useful to verify schedule is running

```

Call `ema.update()` only on real optimiser steps, not on gradient accumulation micro-steps. This is one of the most common bugs in BYOL reimplementations.

2d. Learning Rate Schedule

Both backbones need a warmup + cosine decay schedule. ViT is more sensitive to this than ResNet — skip warmup with ViT and you will likely get unstable early training.

```

def get_scheduler(optimizer, warmup_steps, total_steps):
    def lr_lambda(step):
        if step < warmup_steps:
            return step / warmup_steps    # linear warmup
        progress = (step - warmup_steps) / (total_steps - warmup_steps)
        return 0.5 * (1 + math.cos(math.pi * progress)) # cosine decay
    return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

# ResNet: warmup_steps=1000 is fine
# ViT:    warmup_steps=10000 – ViT needs longer warmup for stability

```

2e. Phase 2 Deliverables

- BYOL forward pass works with both ResNet-18 and ViT-Tiny backbones
- Loss is positive and finite from step 0 for both
- Loss decreases over first 200 steps for both
- EMA tau value logged and verified to be increasing toward 1.0
- No gradient flow through target network (assert `param.grad` is None for all target params)

BYOL holds two full networks in VRAM simultaneously. Without optimisations you will OOM at any useful batch size on a 3050. Each technique below is a real engineering solution applied to a concrete problem.

3a. AMP — Automatic Mixed Precision

```
scaler = torch.cuda.amp.GradScaler()

with torch.autocast(device_type='cuda', dtype=torch.float16):
    loss = model(v1, v2)

scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
scaler.step(optimizer)
scaler.update()

# Log scale — a repeatedly-dropping scale signals FP16 instability
wandb.log({'amp_scale': scaler.get_scale()}, step=step)
```

ViT with AMP: attention scores can overflow FP16 at long sequences. STL-10 at patch_size=8 gives 144 tokens — within safe range. Monitor the loss scale closely for the first 1000 steps of ViT training.

3b. Gradient Accumulation

```
ACCUM = 4    # physical batch 64 -> effective batch 256

for i, (v1, v2) in enumerate(loader):
    with torch.autocast(device_type='cuda', dtype=torch.float16):
        loss = model(v1.cuda(), v2.cuda()) / ACCUM
    scaler.scale(loss).backward()

    if (i + 1) % ACCUM == 0:
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad(set_to_none=True)
        ema.update()    # <-- only here, not every micro-step
```

3c. CPU-Offload Adam

Adam stores two moment tensors per parameter. For ResNet-18 that is ~180 MB of VRAM purely for optimiser state. For ViT-Tiny it is ~90 MB. Offloading to CPU RAM frees this entirely.

```

class CPUOffloadAdam:
    def __init__(self, params, lr=3e-4, betas=(0.9,0.999),
                 eps=1e-8, weight_decay=1e-6):
        self.params = [p for p in params if p.requires_grad]
        self.lr, self.betas, self.eps, self.wd = lr, betas, eps, weight_decay
        self.t = 0
        self.m = [torch.zeros_like(p, device='cpu', dtype=torch.float32)
                  for p in self.params]
        self.v = [torch.zeros_like(p, device='cpu', dtype=torch.float32)
                  for p in self.params]

    @torch.no_grad()
    def step(self):
        self.t += 1
        b1, b2 = self.betas
        for i, p in enumerate(self.params):
            if p.grad is None: continue
            g = p.grad.float().cpu()
            if self.wd: g = g + self.wd * p.data.float().cpu()
            self.m[i] = b1*self.m[i] + (1-b1)*g
            self.v[i] = b2*self.v[i] + (1-b2)*g**2
            m_hat = self.m[i] / (1 - b1**self.t)
            v_hat = self.v[i] / (1 - b2**self.t)
            p.data -= (self.lr * m_hat / (v_hat.sqrt()+self.eps)).to(p.device)

    def zero_grad(self, set_to_none=True):
        for p in self.params:
            if set_to_none: p.grad = None
            elif p.grad is not None: p.grad.zero_()

```

3d. Memory Budget

Configuration	ResNet-18 VRAM	ViT-Tiny VRAM	Batch
Baseline FP32	~3.8 GB	~2.6 GB	32
+ AMP FP16	~2.2 GB	~1.6 GB	64
+ Grad accumulation	~2.2 GB	~1.6 GB	64 phys/256 eff
+ CPU offload Adam	~1.8 GB	~1.3 GB	64 phys/256 eff
+ Grad checkpointing	~1.4 GB	~1.0 GB	64 phys/256 eff

Measure actual values with `torch.cuda.max_memory_allocated()` and record them in `memory_benchmarks.ipynb`.
 These are estimates — your numbers are the ones that go in the notebook.

3e. Phase 3 Deliverables

- VRAM benchmark table with real measurements for BOTH backbones at each stage
- Training stable for 500 steps with all optimisations active, both backbones
- `memory_benchmarks.ipynb` with side-by-side bar charts for ResNet vs ViT
- GradScaler loss-scale logged — not underflowing for either backbone

Three kernels target real bottlenecks in the BYOL pipeline. Read CUDA C Programming Guide chapters 1–4 and parallel reduction patterns before writing any kernel code (~8–10 hrs reading).

4a. Kernel 1 — Fused Gaussian Blur (both backbones)

Applied twice per batch (two augmentation views). The augmentation pipeline is the hottest CPU/GPU path outside the forward pass. A separable implementation — horizontal then vertical pass — halves memory traffic.

```
// kernels/fused_blur/blur_kernel.cu
__global__ void gaussian_blur_h(
    const float* __restrict__ src, float* __restrict__ dst,
    int H, int W, int C,
    const float* __restrict__ kern, int ksize)
{
    extern __shared__ float smem[]; // tile of input row
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y;
    int c = blockIdx.z;
    int tid = threadIdx.x;
    int half = ksize / 2;

    // Load tile into shared memory with halo
    if (tid < blockDim.x + ksize - 1) {
        int xi = min(max((int)(blockIdx.x*blockDim.x + tid - half), 0), W-1);
        smem[tid] = src[c*H*W + y*W + xi];
    }
    __syncthreads();
    if (x >= W) return;

    float val = 0.f;
    for (int k = 0; k < ksize; k++)
        val += kern[k] * smem[tid + k]; // no global re-read
    dst[c*H*W + y*W + x] = val;
}
// Call horizontal kernel, then vertical kernel (transpose indices)
// Benchmark against torchvision.transforms.GaussianBlur at 5 kernel sizes
```

4b. Kernel 2 — Fused Batch Norm

■ ResNet-18

Naive batch norm makes two passes: mean then variance. A fused kernel computes both in one pass using shared memory Welford reduction, halving memory traffic over $N \times H \times W$ elements per channel.

```

// One block per channel, threads cooperate over N*H*W elements
__global__ void fused_bn_forward(
    const float* __restrict__ x,
    float* __restrict__ out,
    const float* __restrict__ gamma, const float* __restrict__ beta,
    float* running_mean, float* running_var,
    int N, int C, int HW, float eps, float momentum)
{
    extern __shared__ float smem[];
    int c = blockIdx.x;
    int tid = threadIdx.x;
    int total = N * HW;

    // Welford online mean+variance (numerically stable)
    float m = 0.f, M2 = 0.f; int count = 0;
    for (int i = tid; i < total; i += blockDim.x) {
        int n = i / HW, hw = i % HW;
        float v = x[n*C*HW + c*HW + hw];
        count++; float delta = v - m;
        m += delta / count;
        M2 += delta * (v - m);
    }
    // Parallel reduction of m and M2 across threads via shared memory
    // ... (reduction omitted for brevity – standard pattern)
    float var = smem_var[0] / total;
    float inv_std = rsqrtf(var + eps);
    // Normalise and scale
    for (int i = tid; i < total; i += blockDim.x) {
        int n = i/HW, hw = i%HW;
        float v = x[n*C*HW + c*HW + hw];
        out[n*C*HW + c*HW + hw] = gamma[c]*(v-smem_mean[0])*inv_std + beta[c];
    }
    // Update running stats
    if (tid == 0) {
        running_mean[c] = (1-momentum)*running_mean[c] + momentum*smem_mean[0];
        running_var[c] = (1-momentum)*running_var[c] + momentum*var;
    }
}

```

4c. Kernel 3 — Fused Layer Norm

■ ViT-Tiny

Layer norm is per-sample, making it simpler than batch norm — each block handles one row (one token or one sample). Same two-pass -> one-pass fusion, but reduction is over embed_dim rather than N*H*W.

```

__global__ void fused_layernorm(
    const float* __restrict__ x,
    const float* __restrict__ gamma, const float* __restrict__ beta,
    float* __restrict__ out, int N, float eps)
{
    extern __shared__ float smem[];
    int row = blockIdx.x;
    int tid = threadIdx.x;
    float val = (tid < N) ? x[row*N + tid] : 0.f;

    // Pass 1: mean
    smem[tid] = val; __syncthreads();
    for (int s = blockDim.x/2; s > 0; s >>= 1) {
        if (tid < s) smem[tid] += smem[tid+s]; __syncthreads(); }
    float mean = smem[0] / N;

    // Pass 2: variance (fused – same shared memory pass)
    smem[tid] = (tid < N) ? (val-mean)*(val-mean) : 0.f; __syncthreads();
    for (int s = blockDim.x/2; s > 0; s >>= 1) {
        if (tid < s) smem[tid] += smem[tid+s]; __syncthreads(); }
    float inv_std = rsqrtf(smem[0]/N + eps);
    if (tid < N)
        out[row*N + tid] = gamma[tid]*(val-mean)*inv_std + beta[tid];
}

```

4d. Pybind11 Bindings and Build

```

// blur.cpp (same pattern for bn.cpp and ln.cpp)
#include <torch/extension.h>
torch::Tensor gaussian_blur_fwd(torch::Tensor input, torch::Tensor kernel);
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("forward", &gaussian_blur_fwd, "Fused separable Gaussian blur"); }

# setup.py
from setuptools import setup
from torch.utils.cpp_extension import BuildExtension, CUDAExtension
setup(name='fused_blur',
      ext_modules=[CUDAExtension('fused_blur',
                                  ['blur.cpp', 'blur_kernel.cu'])],
      cmdclass={'build_ext': BuildExtension})

# Build and install
# python setup.py install
# Then in Python: import fused_blur; out = fused_blur.forward(x, kern)

```

4e. Benchmark Output

Kernel	Input	PyTorch (µs)	Custom (µs)	Speedup	Max diff
Gaussian blur	[64,3,96,96]	measure	measure	target >1.2x	<1e-5
Batch norm	[64,512,6,6]	measure	measure	target >1.1x	<1e-5
Layer norm	[64,144,192]	measure	measure	target >1.1x	<1e-5

Replace 'measure' with your actual Nsight Compute numbers. Honest modest improvements are more credible than suspicious 5x claims on simple ops.

4f. Phase 4 Deliverables

- 3 compiled CUDA extensions with pybind11 bindings
- All kernels numerically correct vs PyTorch (max diff < 1e-5)
- kernel_benchmarks.ipynb with throughput plots and Nsight Compute occupancy
- Gaussian blur kernel integrated into both backbone training pipelines
- Norm kernels integrated into ResNet and ViT training loops respectively

BYOL can silently collapse — loss decreasing while representations are degenerate. The training loss alone tells you nothing. Run these metrics throughout training for both backbones.

5a. kNN Evaluation — run every 2 000 steps

```
@torch.no_grad()
def knn_eval(encoder, train_loader, test_loader, device, k=20):
    encoder.eval()
    feats, labs = [], []
    for x, y in train_loader:
        feats.append(F.normalize(encoder(x.to(device))), dim=-1).cpu())
        labs.append(y)
    bank = torch.cat(feats).T          # [dim, N_train]
    bank_l = torch.cat(labs)
    correct = total = 0
    for x, y in test_loader:
        z = F.normalize(encoder(x.to(device)), dim=-1).cpu()
        sim = z @ bank                # [batch, N_train]
        pred = bank_l[sim.topk(k, dim=-1).indices].mode(dim=-1).values
        correct += (pred == y).sum().item(); total += y.size(0)
    encoder.train()
    return correct / total
```

5b. Linear Probe — run every 10 000 steps

```
def linear_probe(encoder, feat_dim, labeled_loader, test_loader, device, epochs=100):
    encoder.eval()
    def extract(loader):
        fs, ls = [], []
        with torch.no_grad():
            for x, y in loader:
                fs.append(encoder(x.to(device)).cpu()); ls.append(y)
        return torch.cat(fs), torch.cat(ls)
    tr_f, tr_l = extract(labeled_loader)
    te_f, te_l = extract(test_loader)
    clf = nn.Linear(feat_dim, 10)
    opt = torch.optim.Adam(clf.parameters(), lr=1e-3)
    for _ in range(epochs):
        loss = F.cross_entropy(clf(tr_f), tr_l)
        opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        acc = (clf(te_f).argmax(1) == te_l).float().mean().item()
    encoder.train()
    return acc
```

5c. Collapse Detection — run every 500 steps

```
@torch.no_grad()
def collapse_check(encoder, loader, device, n_batches=10):
    encoder.eval()
    feats = []
    for i, (x, _) in enumerate(loader):
        if i >= n_batches: break
        feats.append(F.normalize(encoder(x.to(device)), dim=-1).cpu())
    feats = torch.cat(feats)
    std = feats.std(dim=0).mean().item()
    avg_sim = (feats @ feats.T).fill_diagonal_(0).mean().item()
    encoder.train()
    return {'std': std, 'avg_cosine_sim': avg_sim}
# Healthy: std > 0.1, avg_sim < 0.5
# Collapsed: std < 0.01, avg_sim > 0.95
```

5d. t-SNE Progression

Run at steps 0, 10k, 50k, and final for both backbones. The four-panel progression showing random noise -> class clusters is the most visually compelling output of the entire project.

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

def tsne_plot(encoder, loader, device, step, backbone_name, save_dir):
    feats, labels = extract_features(encoder, loader, device)
    emb = TSNE(n_components=2, perplexity=30, random_state=42)
        .fit_transform(feats.numpy())
    fig, ax = plt.subplots(figsize=(7,6))
    sc = ax.scatter(emb[:,0], emb[:,1], c=labels,
        cmap='tab10', alpha=0.5, s=4)
    plt.colorbar(sc, ax=ax); ax.axis('off')
    ax.set_title(f'{backbone_name} - step {step:,}')
    fig.savefig(f'{save_dir}/tsne_{backbone_name}_{step:07d}.png', dpi=150)
    plt.close()
```

5e. Checkpoint Format

```
def save(model, optimizer, scaler, ema, step, metrics, path):
    torch.save({
        'step': step, 'metrics': metrics,
        'encoder': model.online_enc.state_dict(), # for eval - load alone
        'byol': model.state_dict(),
        'optimizer': optimizer.state_dict(),
        'scaler': scaler.state_dict(),
        'ema_step': ema.step,
    }, path)
# Keep only last 5 checkpoints to save disk space
```

5f. Phase 5 Deliverables

- kNN accuracy curve from step 0 to end of training for BOTH backbones — must show upward trend
- Linear probe accuracy logged for both backbones
- t-SNE plots at 4 checkpoints for both backbones — 8 plots total
- Collapse detection running every 500 steps — std and avg_cosine_sim logged
- Checkpoint resume verified for both backbone configs

Run ablations on the simpler backbone first (ResNet-18) to keep iteration fast. Repeat the most interesting ones (no_predictor, no_momentum) on ViT-Tiny to see if collapse dynamics differ across architectures — they sometimes do.

Experiment	Change	Expected outcome	Run on
byol_baseline	None	kNN improves to ~60–70%, representations cluster in t-SNE	Both
no_predictor	Remove predictor head from online network	Immediate collapse — kNN stays at ~10% (chance)	Both
no_momentum	tau=0 (direct weight copy each step)	Collapse or severe instability within 5k steps	Both
low_momentum	Fixed tau=0.9 instead of cosine schedule	Slower convergence, lower final kNN	ResNet
no_aug_asymmetry	Same augmentation probabilities for both views	Slightly lower final kNN — subtle but real	ResNet
fp32_vs_fp16	Disable AMP	Same kNN, higher VRAM, slower wall-clock	Both
resnet_vs_vit	Same BYOL config, swap backbone	Compare final kNN and convergence speed	Both — this is the backbone comparison

Run each ablation for 20 000 steps — sufficient to confirm collapse or not. The backbone comparison (last row) uses your full 200-epoch runs.

6a. Collapse Detection in Ablations

```
# During no_predictor and no_momentum runs:
# Stop training early if collapse is confirmed — no need to continue
def is_collapsed(metrics):
    return metrics['std'] < 0.01 or metrics['avg_cosine_sim'] > 0.9

for step, (v1, v2) in enumerate(loader):
    # ... training step ...
    if step % 500 == 0:
        m = collapse_check(model.online_enc, eval_loader, device)
        wandb.log({'collapse/std': m['std'],
                  'collapse/avg_sim': m['avg_cosine_sim']}, step=step)
        if is_collapsed(m):
            print(f"Collapse confirmed at step {step}. Stopping.")
            break
```

6b. ResNet vs ViT Comparison

This is the most valuable ablation in the project. Same BYOL outer shell, same dataset, same augmentations, same memory optimisations — only the backbone differs. Compare:

- kNN accuracy at each checkpoint (plot both curves on one figure)

- Linear probe final accuracy
- Wall-clock time per epoch
- Peak VRAM
- Convergence speed — how many steps to reach 50% kNN?
- Collapse behaviour in no_predictor ablation — do they differ?

6c. Phase 6 Deliverables

- 6 ablation runs complete with kNN logged every 2 000 steps
- Collapse confirmed in no_predictor and no_momentum for both backbones
- ablations.ipynb with all runs on a single overlaid plot
- ResNet vs ViT comparison plot and written analysis
- 200-word written analysis explaining why each ablation behaves as it does

Repository Structure

```
byol-from-scratch/
  kernels/
    fused_blur/
      blur_kernel.cu      # Separable Gaussian blur – used by both backbones
      blur.cpp            # pybind11 bindings
      setup.py
      benchmark.py
    fused_batchnorm/
      bn_kernel.cu        # Fused batch norm – ResNet-18 only
      bn.cpp
      setup.py
      benchmark.py
    fused_layernorm/
      ln_kernel.cu        # Fused layer norm – ViT-Tiny only
      ln.cpp
      setup.py
      benchmark.py
  data/
    download.py           # STL-10 download and extraction
    augmentations.py      # BYOL asymmetric two-view pipeline
    dataset.py            # STL10Unlabeled and STL10Labeled
    prefetch.py           # Pinned memory async prefetch wrapper
  model/
    resnet.py             # ResNet-18 from scratch
    vit.py                # ViT-Tiny from scratch
    byol.py               # Backbone-agnostic BYOL wrapper
    ema.py                # EMAUpdater with cosine momentum schedule
    mlp.py                # Shared projector/predictor MLP
  training/
    train.py              # Main loop – takes --backbone resnet|vit
    optimizer.py          # CPUOffloadAdam
    scheduler.py          # Warmup + cosine decay
    checkpointing.py       # Save, load, keep-last-N
  evaluation/
    knn_eval.py
    linear_probe.py
    collapse_check.py
    visualize.py          # t-SNE plots, feature std plots
  notebooks/
    kernel_benchmarks.ipynb # Throughput for all 3 kernels
    memory_benchmarks.ipynb # VRAM table for both backbones
    representation_quality.ipynb # kNN curves + t-SNE progression
    ablations.ipynb        # All ablations overlaid
    resnet_vs_vit.ipynb     # Head-to-head backbone comparison
  configs/
    byol_resnet18.yaml
    byol_vit_tiny.yaml
    ablation_no_predictor.yaml
    ablation_no_momentum.yaml
    ablation_low_momentum.yaml
    ablation_no_aug_asymmetry.yaml
    ablation_fp32.yaml
  results/
    checkpoints/          # Saved every 2 000 steps, keep last 5
    plots/                # t-SNE PNGs, loss curves
  README.md              # t-SNE progressions, results table, backbone comparison
  requirements.txt
```

Skills Demonstrated

Skill	Technique	Where
CUDA programming	Parallel reductions, shared memory tiling, warp ops, Welford numerics	All 3 kernels
CUDA tooling	Nsight Compute — occupancy, memory BW, stall reasons	Kernel benchmarks
Memory management	AMP FP16, grad accumulation, CPU-offload Adam, grad checkpointing	Phase 3
PyTorch internals	GradScaler mechanics, autocast, set_to_none, no_grad scoping	Training loop
Architecture	ResNet residual blocks AND ViT patch embedding + attention from scratch	Phase 1
Self-supervised ML	BYOL theory, EMA target, predictor asymmetry, collapse prevention	Phases 2+6
Evaluation	kNN probe, linear probe, t-SNE, collapse detection	Phase 5
Experiment design	Controlled single-variable ablations, honest measurement	Phase 6
Comparative analysis	ResNet vs ViT on same task with same training infrastructure	Phase 6

Timeline

Assumes 8–10 hours/week on a laptop. Sessions of 2–3 hours are sufficient for all development. Long training runs happen unattended.

Week	Phase	Key Deliverable	Hrs
1	ResNet-18	Backbone implemented, supervised smoke test passes, STL-10 loader correct	10–12
2	ViT-Tiny	ViT-Tiny implemented, same smoke test passes, both backbones share forward interface	10–12
3	BYOL Core	BYOL wrapper works with both backbones, EMA updater with cosine schedule, LR warmup	12–14
4	Memory Opts	AMP + grad accum + CPU-offload Adam for both backbones. VRAM benchmark table produced.	12–15
5–6	CUDA Kernels	Gaussian blur, fused batch norm (ResNet), fused layer norm (ViT) compiled, benchmarked, integrated.	22–30
6–7	Evaluation	kNN, linear probe, collapse detection, t-SNE running. Full training run for both backbones.	10–12
7–8	Ablations	6 ablations complete. Collapse confirmed. ResNet vs ViT comparison notebook finalised.	10–12

	Hours
Total implementation	76–107 hrs
At 8 hrs/week	10–13 weeks
At 10 hrs/week	8–11 weeks
Cloud wrap-up cost	~\$2–5 (Vast.ai A10, 2–4 hrs)

Portfolio Narrative

"I implemented BYOL from scratch with two backbone architectures — ResNet-18 and ViT-Tiny — both written without library imports. I wrote three custom CUDA kernels (fused Gaussian blur, fused batch norm, fused layer norm), implemented AMP mixed-precision training, gradient accumulation, and a CPU-offload Adam optimiser. I tracked representation quality throughout with kNN and linear probe evaluation, detected collapse quantitatively, and ran controlled ablations removing the predictor head and momentum encoder to directly verify the theoretical claims of the BYOL paper. The ResNet and ViT results are compared head-to-head under identical training conditions."

What each claim signals

Claim	Signal
Two backbones, no library imports	You understand both conv and attention architectures
3 custom CUDA kernels	You can go below PyTorch when it matters
CPU-offload Adam	You understand optimiser state as a memory cost
kNN and linear probe throughout	You know how to evaluate self-supervised learning honestly
Collapse detection quantitatively	You know what failure looks like, not just success
Controlled ablations	You understand why BYOL works, not just that it runs
ResNet vs ViT comparison	You can make and communicate architectural tradeoffs

Two architectures. Three kernels. One clean story.